

Sumário:

- 1 - Cronograma das Aulas e Assuntos
- 2 - Resumos por Aula + Pontos importantes
- 3 - TPs (em ordem)
- 4 - Provas simuladas
- 5 - Resumo de tudo

1 - Cronograma das Aulas e Assuntos

Aula 1: O que é Engenharia de Software 3? O que é Arquitetura de Software?

Em Engenharia 3, procuramos entender como os componentes se comportam dentro da estrutura da aplicação que estamos construindo. Ela é uma continuação das segmentações de Engenharia de Software vistas anteriormente.

- Em ES 1, vimos sobre os conceitos mais básicos, processos ágeis, camadas e macroáreas.
- Em ES 2, vimos sobre a análise da engenharia de requisitos e o padrão de documentação UML.

Aula 2: Revisão de Diagramas UML

Revisamos os elementos mais básicos da UML, também aqueles aprendidos em ES 1, como a Matriz de Requisitos Funcionais e Não Funcionais, Diagramas de Casos de Uso, Documento de Especificação de Caso de Uso, Diagrama de Fluxo, Diagrama de

Aula 3: Elementos da Arquitetura de Software

Aula 4: Padrões de Arquitetura de Software e UML

Aula 5: Microserviços e SOA (ERP e SAP)

Aula 6: Estilos de Arquitetura

Aula 7: Modelos de Armazenamento

2 - Resumos por Aula + Pontos importantes

Aula 1: O que é Engenharia de Software 3? O que é Arquitetura de Software?

Este resumo apresenta os conceitos fundamentais de Engenharia de Software III, baseados na bibliografia recomendada de Pressman e Sommerville.

Definição de Arquitetura de Software

A arquitetura de um sistema é o arcabouço abrangente que descreve sua forma, estrutura, componentes e suas articulações. O padrão ISO/IEEE 1471-2000 define-a como a organização fundamental de um sistema incorporada em seus componentes, seus relacionamentos com o ambiente e os princípios que conduzem seu design e evolução.

A arquitetura possui um duplo propósito:

- * Apoiar a realização de objetivos de negócio através de uma mistura equilibrada de recursos.
- * Utilizar uma combinação ótima de tecnologias de implementação em hardware, software e operações.

O Papel do Arquiteto de Software

O arquiteto de software coordena artefatos técnicos e estabelece a estrutura geral de cada visão arquitetural. Este papel exige uma visão ampla, não detalhada, e a capacidade de decidir sobre tecnologias, frameworks e infraestrutura.

Vantagens e Modelos

Projetar e documentar a arquitetura traz diversas vantagens, tais como:

- * Tratamento da qualidade do sistema e auxílio à gestão da complexidade.
- * Guia para o consenso e suporte ao planejamento.
- * Preservação da integridade dos módulos e reuso em larga escala.

Os ****modelos arquiteturais**** são utilizados para representar diferentes visões da arquitetura, como decomposições em módulos ou visões específicas de estrutura e comportamento.

Desafios e Requisitos Não Funcionais

O desenvolvimento de software enfrenta problemas de complexidade, conformidade, mutabilidade e invisibilidade. Ao projetar a arquitetura, é essencial considerar requisitos não funcionais, como:

- * **Desempenho:** Priorização de operações críticas.
- * **Proteção e Segurança:** Uso de estruturas em camadas e centralização de operações de segurança.
- * **Disponibilidade:** Inclusão de redundância.
- * **Manutenção e Distribuição:** Uso de componentes de baixa granularidade.

Aula 2: Elementos da Arquitetura de Software

Esta estrutura apresenta um resumo dos tópicos abordados na aula de Engenharia de Software III sobre arquitetura de sistemas.

Conceitos de Arquitetura de Software

- * Um estilo de arquitetura é definido como um padrão de organização de um sistema.
- * A escolha da organização reflete a estratégia básica utilizada para estruturar o software.

Sistemas Distribuídos

- * São sistemas nos quais componentes interligados em rede se comunicam e coordenam suas ações exclusivamente por meio de troca de mensagens.
- * A motivação para seu uso inclui compartilhamento de recursos, interligação em rede, computação móvel e a oferta de serviços via internet.

Modelo Cliente-Servidor

- * Neste modelo, o sistema é organizado em servidores, que oferecem serviços (ex: banco de dados, arquivos, impressão), e clientes, que solicitam e utilizam esses serviços.
- * A comunicação ocorre via rede, frequentemente utilizando chamadas remotas de procedimento.
- * **Variações de clientes:**
 - * **Thin Client (Cliente fino):** O processamento e gerenciamento de dados ocorrem no servidor, deixando ao cliente apenas a tarefa de apresentação.
 - * **Fat Client (Cliente gordo):** O cliente executa a maior parte do processamento e a lógica de aplicação, deixando ao servidor apenas o gerenciamento de dados.

Modelo em Camadas (Tier)

- * **Monolíticas:** Aplicações antigas onde toda a lógica e dados residiam em uma única máquina.
- * **2 Tier:** Separa o banco de dados em um servidor, mas mantém a lógica no cliente, gerando problemas de gerenciamento de versões.

* **3 Tier:** Padrão para sistemas corporativos Web, separando a camada de apresentação, lógica de negócio e persistência. Promove modularidade, baixo acoplamento e facilita a manutenção.

* **MVC (Modelo-Visão-Controle):** Um framework para organizar camadas, onde o *Model* lida com dados, a *View* com a interface e o *Controller* gerencia o fluxo e comportamento da aplicação.

Aula 3: Revisão de Diagramas UML

Este documento serve como uma revisão abrangente da Linguagem de Modelagem Unificada (UML) versão 2.5.1 para o curso de Engenharia de Software III. Classifica os diagramas UML em dois tipos principais — estruturais e comportamentais — e fornece descrições detalhadas e exemplos para cada um.¹²

Diagramas Estruturais

Esses diagramas focam na estrutura estática do sistema.³

Diagrama de Pacotes: Organiza modelos de sistema (como classes e casos de uso) em namespaces para fornecer estruturação e organização genéricas.⁴

Diagrama de Classes: Representa a estrutura estática dos objetos, incluindo seus atributos, métodos e relacionamentos (multiplicidades).⁵

Diagrama de Objetos: Complementa o diagrama de classes mostrando o conteúdo específico dos atributos dos objetos (instâncias) em um determinado momento durante a execução do sistema.⁶

Diagrama de Componentes: Mostra a organização e a fiação dos componentes físicos ou lógicos e seus inter-relacionamentos.⁷

Diagrama de Implantação: Representa a arquitetura física e a configuração do sistema, mapeando componentes de software para hardware.⁸

Diagrama de Estrutura Composta: Descreve a colaboração interna de classes, interfaces ou componentes que cooperam para executar uma funcionalidade específica.⁹

Diagramas Comportamentais

Esses diagramas focam nos aspectos dinâmicos do sistema. o sistema e como ele muda ao longo do tempo.¹⁰

Diagrama de Casos de Uso: Ilustra os atores, os casos de uso e os relacionamentos entre eles para mostrar a funcionalidade do sistema.¹¹

Diagrama de Máquina de Estados: Especifica a sequência de estados pelos quais um objeto passa, incluindo eventos e condições que desencadeiam mudanças de estado.¹²

Diagrama de Atividades: Modela o comportamento do sistema como um fluxo de processos, incluindo caminhos lógicos baseados em condições e processamento concorrente.¹³

Diagrama de Sequência: Concentra-se na ordem cronológica de eventos e trocas de mensagens entre objetos em um eixo horizontal e vertical.¹⁴

Diagrama de Comunicação: Semelhante a um diagrama de sequência, mas concentra-se na visualização dos relacionamentos entre instâncias de objetos cooperantes, em vez de apenas no tempo.¹⁵

Diagrama de Visão Geral da Interação: Funde diagramas de atividades e de sequência para combinar fragmentos de interação com fluxos de decisão.¹⁶

Diagrama de Tempo: Define o comportamento dos objetos dentro de uma escala de tempo específica, mostrando as mudanças de estado ao longo do tempo.¹⁷

O documento conclui com uma tarefa de Estudo de Caso que exige que os alunos identifiquem 15 requisitos funcionais e 10 não funcionais, criem uma rastreabilidade elabore uma matriz e desenvolva um diagrama de casos de uso para um projeto de software proposto.

Aula 4: Padrões de Arquitetura de Software e UML

Aula 5: Microserviços e SOA (ERP e SAP)

Este documento apresenta uma visão detalhada da Arquitetura Orientada a Serviços (SOA) e sua aplicação na Engenharia de Software.

Aqui estão os pontos principais:

O que é SOA?

Definição: É um estilo de arquitetura focado nos processos e funções de negócio, implementando-os através de serviços reutilizáveis e interoperáveis.¹

O que é um Serviço: É a menor unidade de uma aplicação SOA, representando uma funcionalidade de negócio bem definida (ex: verificar estoque ou crédito).²

Perspectivas: A interpretação de SOA varia conforme o papel: para um Engenheiro de Software, é uma arquitetura baseada em padrões abertos para integração; para um Desenvolvedor, é um framework baseado em web services (SOAP/XML) para invocação remota.³

Características e Princípios de Design

De acordo com Thomas Erl, um serviço deve seguir oito regras fundamentais:

Reutilização e Composição: Serviços devem ser reutilizáveis e capazes de se compor para resolver problemas maiores.4

Autonomia e Baixo Acoplamento: Devem ser independentes e permitir mudanças internas sem afetar o consumidor.5

Contrato Formal e Abstração: Devem possuir um contrato padronizado que informe o que o serviço faz, funcionando como uma "caixa preta" que esconde sua lógica interna.6

Ausência de Estado e Descoberta: Devem evitar a alocação de recursos por longos períodos (stateless) e ser facilmente localizados em diretórios.7

Servidores Proxy e APIs

Proxy: Age como um intermediário entre uma rede interna e a internet, sendo usado para compartilhar conexões e bloquear páginas.8

APIs: São interfaces que permitem a integração de tecnologias via protocolo HTTP, sendo essenciais para a comunicação em microserviços.9

Tipos de API: Podem ser Privadas (processos internos), Restritas (parceiros de negócio) ou Públicas (abertas a qualquer desenvolvedor).10

Conclusões sobre a Implementação

SOA é considerada uma filosofia arquitetural e uma linha de pensamento, não apenas uma tecnologia ou produto que se compra.11

Vantagens: Oferece baixo acoplamento, alta interoperabilidade entre plataformas e facilidade de testes.12

Deve estar alinhada ao planejamento estratégico da empresa para integrar sistemas como ERPs e CRMs.13

◆ 1. O que é SOA (Service-Oriented Architecture)

É um estilo de arquitetura de software baseado em serviços.

Foco: processos de negócio da empresa.

Os sistemas são construídos como um conjunto de serviços reutilizáveis e interoperáveis.

👉 Serviço:

Unidade básica da SOA.

Representa uma função de negócio (ex: registrar pedido, verificar estoque).

◆ 2. Ideia central da SOA

Quebrar sistemas em serviços menores

Combinar esses serviços conforme a necessidade

Reutilizar serviços em novos contextos

◆ 3. Características principais da SOA

Comunicação baseada em mensagens

Baixo acoplamento (serviços independentes)

Abstração (não importa como foi implementado)

Interoperabilidade entre diferentes tecnologias

Uso de padrões (ex: XML, web services)

◆ 4. Vantagens da SOA

Reutilização de código

Facilidade de integração entre sistemas

Flexibilidade e adaptação ao negócio

Facilidade de testes

Suporte a múltiplas plataformas

◆ 5. O que SOA NÃO é

✗ Não é uma tecnologia

✗ Não é uma metodologia

✗ Não é um webservice

✓ É uma filosofia/estilo arquitetural

◆ 6. Princípios importantes dos serviços (Thomas Erl)

Os mais cobrados:

Serviços são reutilizáveis

Serviços têm baixo acoplamento

Serviços são autônomos

Serviços possuem contrato formal

Serviços abstraem a lógica

Serviços podem ser compostos

◆ 7. APIs dentro da SOA

APIs permitem a comunicação entre sistemas via HTTP.

São o principal meio de integração (especialmente em microserviços).

Tipos de APIs:

Privadas → uso interno

Restritas → parceiros
Públicas → abertas a desenvolvedores

◆ 8. Microserviços (introdução)

Evolução da SOA.

Sistema dividido em serviços pequenos e independentes.

Comunicação geralmente via APIs.

◆ 9. Arquitetura em N-camadas

Sistema dividido em camadas (ex: UI → API → Serviços → Banco).

“N” = número variável de camadas.

Vantagens:

Organização

Manutenção fácil

Escalabilidade

Segurança

◆ 10. EDA vs Fila com Worker

Fila (Worker) → foco em tarefas específicas

EDA (Event-Driven) → foco em eventos (vários sistemas reagem ao mesmo evento)

✅ RESUMO FINAL (para prova)

SOA = arquitetura baseada em serviços reutilizáveis

Serviço = função de negócio

Características: baixo acoplamento, interoperabilidade, abstração

APIs = meio de comunicação

Microserviços = evolução da SOA

N-camadas = separação por responsabilidades

EDA = arquitetura baseada em eventos

Aula 6: Estilos de Arquitetura

Claro! Segue um resumo do texto sobre **Estilos de Arquitetura de Software**:

Resumo do Texto: Estilos de Arquitetura

Um **estilo de arquitetura** é uma família de arquiteturas com características comuns (ex.: N camadas, microserviços). Cada estilo impõe restrições que moldam o design e trazem benefícios específicos, mas também desafios.

Principais estilos abordados:

1. ****N camadas****

- Divisão em camadas lógicas e físicas (apresentação, negócios, dados).
- Ideal para migração de sistemas legados para a nuvem.
- Desvantagem: mudanças afetam várias partes, limitando agilidade.

2. ****Web-Queue-Worker****

- Front-end + fila + worker para tarefas pesadas/assíncronas.
- Fácil implantação no Azure, escalabilidade independente.
- Risco de tornar-se monolítico se mal projetado.

3. ****Microserviços****

- Serviços pequenos, autônomos, com dados próprios.
- Alta agilidade, escalabilidade e isolamento.
- Exige maturidade em DevOps e gerencia alta complexidade.

4. ****Arquitetura orientada a eventos****

- Produtores e consumidores desacoplados via eventos.
- Excelente para tempo real (IoT, finanças).
- Desafios: consistência eventual, ordenação e entrega de eventos.

5. ****Big Data****

- Processa grandes volumes (batch e streaming).
- Usa data lakes, processamento paralelo e análise em tempo real.
- Essencial para insights preditivos e IoT.

6. ****Computação de grande escala****

- Milhares de núcleos para tarefas intensivas (simulações, modelagem).
- Divide trabalho em tarefas paralelas ou fortemente acopladas.
- Usa serviços como Azure Batch.

Considerações importantes:

- Restrições dos estilos trazem propriedades desejáveis, mas também desafios (complexidade, latência, consistência eventual, gerenciamento).
- A escolha deve ser baseada no domínio do problema, nos drivers de negócio e nos requisitos não funcionais (ex.: tempo de mercado, custo).
- Não se trata de pureza arquitetônica, mas de soluções práticas. O estilo pode ser ajustado conforme necessário.

Aula 7: Modelos de Armazenamento

Claro! Aqui está um resumo do texto sobre **Modelos de Dados**:

Resumo do Texto: Entendendo Modelos de Dados

Soluções modernas lidam com dados diversos (transações, eventos, documentos, etc.). Um único banco de dados raramente atende a todos os padrões de acesso. Por isso, adota-se **persistência poliglota** – usar múltiplos modelos de armazenamento.

Etapas para selecionar modelos de dados:

1. Identificar padrões de acesso da carga de trabalho.
2. Mapear padrões para modelos de armazenamento.
3. Listar serviços do Azure que implementam esses modelos.
4. Aplicar critérios (consistência, latência, custo, etc.).
5. Combinar modelos apenas quando os padrões ou ciclos de vida divergem claramente.

Principais modelos de dados e usos:

Modelo	Finalidade	Exemplos de serviços Azure
Relacional (OLTP)	Transações consistentes	SQL Database, PostgreSQL, MySQL
Documento	Esquema flexível, JSON	Cosmos DB (NoSQL), DocumentDB
Família de colunas	Alta ingestão de gravação, dados esparsos	Cassandra gerenciada, HBase, Data Explorer
Chave-valor	Baixa latência, cache	Redis gerenciado, Cosmos DB (Tabela/NoSQL)
Grafo	Relações complexas	Extensões de grafo do SQL Server
Série temporal	Métricas com carimbo de data/hora	Azure Data Explorer, Eventhouse (Fabric)
Objeto/arquivo	Blobs, arquivos grandes	Blob Storage, Data Lake Storage
Pesquisa/indexação	Texto completo, relevância	Azure AI Search
Vetorial	Similaridade semântica (ANN)	Azure AI Search, Cosmos DB
Análise/OLAP	BI, agregação histórica	Fabric, Data Explorer, Databricks

Características comparativas (não relacionais):

- **Documento**: esquema em leitura, atomicidade por documento, ideal para agregações.
- **Família de colunas**: dados largos e esparsos, alta taxa de gravação.
- **Chave-valor**: busca por chave, latência baixíssima, valor opaco.
- **Grafo**: relacionamentos normalizados, travessias profundas.
- **Série temporal**: apenas acréscimo, consultas por janela de tempo.
- **Objeto**: operações no objeto inteiro, escala praticamente ilimitada.
- **Pesquisa**: indexação invertida, consultas de texto completo.

Heurística de escolha:

- Transações estritas → **Relacional**
- JSON/agregações evolutivas → **Documento**
- Baixa latência/cache → **Chave-valor**
- Telemetria intensa → **Família de colunas** ou **Série temporal**
- Relacionamentos profundos → **Grafo**
- Grandes binários/data lake → **Objeto**
- Busca por relevância → **Pesquisa/indexação**
- Métricas com timestamp → **Série temporal**
- Similaridade semântica → **Vetorial**

Antipadrões a evitar:

- Múltiplos microsserviços compartilhando um mesmo banco de dados.
- Adicionar modelo sem maturidade operacional.
- Usar índice de busca como repositório primário.

Quando reavaliar a escolha:

- Muitas junções em repositório de documentos → migrar para relacional.
- Alta CPU em índice de busca → transferir para motor de análise.
- Documentos grandes com contenção de atualização → remodelar agregações.
- Consultas lentas em colunas → adotar série temporal.
- Latência alta em grafos → adicionar visões materializadas.

3 - TPs (em ordem)

TP 1

ES3-Aula_01_questionário (10 Pontos)

1.O que é arquitetura de software segundo o padrão ISO/IEEE 1471-2000?Requer resposta. Opção única.

(1 Ponto)

- a) A estrutura de dados utilizada para armazenar informações.
- b) A organização fundamental de um sistema, incorporada em seus componentes e seus relacionamentos.
- c) Um modelo de interface gráfica para o usuário.
- d) Um conjunto de algoritmos que definem o comportamento do sistema.

2.Quais são os dois propósitos principais da arquitetura de software segundo o Software Engineering Institute?Requer resposta. Opção única.

(1 Ponto)

- a) Definir a interface do usuário e o banco de dados.
- b) Projetar recursos para objetivos de negócio e usar tecnologias de implementação.
- c) Criar diagramas UML e modelos de dados.
- d) Desenvolver testes automatizados e documentação técnica.

3.Qual a diferença entre arquitetura de software e design de software?Requer resposta. Opção única.

(1 Ponto)

- a) A arquitetura define a estrutura geral e o design detalha a implementação dos componentes.
- b) A arquitetura é de baixo nível e o design de alto nível.
- c) Não há diferença entre arquitetura e design.
- d) A arquitetura trata de detalhes de implementação e o design de decisões estratégicas.

4.Cite uma vantagem de documentar a arquitetura de software.Requer resposta. Opção única.

(1 Ponto)

- a) Evita a criação de diagramas.
- b) Reduz a segurança do sistema.
- c) Facilita a comunicação entre a equipe.
- d) Aumenta a complexidade do sistema.

5.O que é um modelo na arquitetura de software?Requer resposta. Opção única.

(1 Ponto)

- a) Uma abstração semântica que representa uma simplificação da realidade.
- b) Um conjunto de testes automatizados.
- c) Uma estrutura de banco de dados.
- d) Um algoritmo de autenticação.

6.Quais são os tipos de modelos utilizados na arquitetura de software?Requer resposta. Opção única.

(1 Ponto)

- a) Modelos de interface e modelos de segurança.
- b) Modelos estruturais e modelos comportamentais.
- c) Modelos de dados e modelos de rede.
- d) Modelos gráficos e modelos textuais.

7.O que é uma visão na arquitetura de software?Requer resposta. Opção única.

(1 Ponto)

- a) Uma representação gráfica de um sistema completo.
- b) Parte do modelo voltado para um único aspecto do sistema.
- c) Um conjunto de algoritmos de segurança.
- d) Uma estrutura de banco de dados relacional.

8.Qual é o objetivo da visualização de casos de uso na arquitetura 4+1?Requer resposta. Opção única.

(1 Ponto)

- a) Representar a distribuição física do sistema.
- b) Planejar o conteúdo técnico de iterações.
- c) Organizar os pacotes em camadas.
- d) Descrever tarefas e encadeamentos.

9.Qual é o foco da visualização lógica na arquitetura 4+1?Requer resposta. Opção única.

(1 Ponto)

- a) Distribuição física do sistema.
- b) Organização de tarefas e processos.
- c) Organização das classes de design em pacotes e subsistemas.
- d) Compreensão da estrutura e organização do design do sistema.

10.O que é arquitetura de referência?Requer resposta. Opção única.

(1 Ponto)

- a) Um conjunto de algoritmos de autenticação.
- b) Um padrão predefinido testado em contextos específicos.

- c) Uma estrutura de banco de dados relacional.
- d) Um modelo de interface gráfica.

TP 02

1. O que é um estilo de arquitetura segundo Garlan e Shaw? * (1 Ponto)

Um conjunto de ferramentas de desenvolvimento

Um padrão de organização do sistema

Um tipo de linguagem de programação

Um modelo de banco de dados

2. Qual é a principal característica de um sistema distribuído? * (1 Ponto)

Executa em um único computador

Utiliza banco de dados relacional

Componentes se comunicam por mensagens

É baseado em arquivos locais

3. Qual é uma vantagem do modelo cliente-servidor? * (1 Ponto)

Dificuldade de expansão

Arquitetura monolítica

Facilidade de expansão do sistema

Ausência de rede

4. No modelo cliente-servidor, o que caracteriza um 'Thin Client'? * (1 Ponto)

Executa toda a lógica da aplicação

Gerencia os dados localmente

Executa apenas a interface de apresentação

Não se conecta à rede

5. Qual é a principal vantagem do modelo em camadas? * (1 Ponto)

Dificuldade de manutenção

Desenvolvimento incremental

Baixa modularidade

Alta dependência entre componentes

6. O que motivou a transição para o modelo de três camadas? * (1 Ponto)

Redução de custos com hardware

Popularização de sistemas monolíticos

Necessidade de portabilidade e flexibilidade

Eliminação da camada de apresentação

7. Qual é a função da camada 'Model' no padrão MVC? * (1 Ponto)

Exibir os dados ao usuário

Interpretar ações do usuário
Representar os dados e lógica da aplicação
Controlar a interface gráfica

8. No padrão MVC, quem é responsável por interpretar ações do usuário? * (1 Ponto)

Model

View

Controller

O Servidor

9. Qual é uma característica do modelo de três camadas? * (1 Ponto)

Alta acoplagem entre componentes

Separação entre lógica de negócio e apresentação

Execução em um único nó físico

Ausência de modularidade

10. O padrão MVC promove qual das seguintes práticas? * (1 Ponto)

Acoplamento forte

Redução da modularidade

Separação de responsabilidades

Centralização de lógica na View

TP 03

1. Muitas organizações estão procurando reorganizar os recursos de informações como capacidades funcionais substancialmente independentes e reutilizáveis, que criam um ambiente inerentemente adaptável. Através da descrição dessas capacidades reutilizáveis utilizando protocolos padrão e abertos, uma organização pode criar os serviços auto-explicativos que podem ser utilizados independentemente da tecnologia de base.

A independência técnica permite que os serviços sejam utilizados em contextos diferentes, para atingir a padronização de processos, regras e políticas de negócios. O texto se refere a qual abordagem da arquitetura para desenvolvimento de sistemas: * (1 Ponto)

SOA (Service-oriented Architecture);

UDDI (Universal Description, Discovery and Integration)

SODA (Service-Oriented Development of Applications)

nenhuma das alternativas.

2. Os princípios "Descoberta e Descrição" e "Manutenção Rápida de Aplicativo" referem-se ao ciclo de vida de serviços e como são localizados, aplicados, desenvolvidos e mantidos. Os fornecedores de ferramentas estão oferecendo cada vez mais formas de armazenar, catalogar, procurar e recuperar serviços. O suporte para evolução contínua de serviços é um aspecto crítico disso, que conduz a várias versões de serviços. Esse texto se refere a: *

Os serviços são facilmente gerenciáveis dentro do ambiente de uma arquitetura de microserviços;

O gerenciamento dos serviços, controle de versão e integridade de dados são desafios para a área de tecnologia da empresa, por isso precisamos de ferramentas nesse tipo de arquitetura;

Devemos adotar a arquitetura de microserviços mesmo que nossa equipe não tenha experiência e uma cultura DEVOPS madura para esse tipo de arquitetura;

3. Um serviço é uma implementação de uma bem definida funcionalidade de negócio. É a menor unidade de uma aplicação SOA. Assinale a alternativa incorreta: * (1 Ponto)

Cada serviço é uma base de código separado, que pode ser gerenciado por uma equipe de desenvolvimento pequena.

Os serviços comunicam-se entre si por meio de APIs bem definidas. Detalhes da implementação interna de cada serviço ficam ocultos de outros serviços.

Uma equipe pode atualizar um serviço existente, mas tem que recompilar e reimplantar o aplicativo inteiro.

4. Para implementar uma arquitetura SOA - Arquitetura orientada a serviços, precisamos de uma plataforma que ofereça os seguintes recursos básicos em suas ferramentas:

Apresentação dos serviços, colaboração entre os serviços, interação dos serviços,

processamento dos serviços, dados dos serviços, monitoramento e segurança, orquestração dos serviços.

Somente o armazenamento e orquestração dos serviços é necessária.

Serviços podem ser publicados ou hospedados em diferentes tipos de hosts, não necessitam de segurança e controle de versão, nem de plataformas adequadas para serem implementados;

5. Sobre as APIS - Interfaces de Programação de Aplicativos. Assinale a alternativa incorreta:

As APIs costumam ser o meio pelo qual os microsserviços se comunicam. Além disso, podem ser usadas para expor os dados e as funcionalidades de uma aplicação a terceiros, abrindo caminho para integrações poderosas.

Possibilitam a integração de tecnologias que operam a base de um protocolo HTTP. Seu objetivo é promover a comunicação entre aplicações que precisam se integrar.

Nas APIS os eventos são gravados em um log. Eventos são estritamente ordenados (dentro de uma partição) e duráveis.

6. As dificuldades na construção de soluções de software em escala corporativa surgem de, pelo menos, quatro origens principais desafios: Entender domínios de negócio complexos, avaliar o uso mais eficiente dos recursos de TI, gerenciar o esforço de desenvolvimento, implementar soluções para uma seleção complicada de tecnologias de infra-estrutura e ferramentas diversas de software. Neste contexto qual a melhor estratégia a ser escolhida pelas empresas para diminuir os riscos: * (1 Ponto)

Aumentando níveis de colaboração dentro das equipes e através de funções de profissionais.

Desenvolver soluções e uma arquitetura de software, flexível, que permitir reutilizar esforços existentes que implementam rapidamente as funcionalidades de negócios, reorganizar os recursos de informações independentes e reutilizáveis, criar um ambiente adaptável a diversas tecnologias e integrações.

O uso de uma abordagem orientada a objetos é projetada para ajudar a resolver esse intervalo entre os analistas de negócios e os especialistas da linha de negócios e especialistas IT, como por exemplo arquitetos, analistas de sistemas, integradores, designers e desenvolvedores.

7. " A tecnologia SOA tem como foco atender o processo de negócio da empresa", explique com suas palavras o que isso representa para o gerente de projetos que utiliza SOA, e para o Diretor de uma área de negócio. * (1 Ponto)

O SOA é uma metodologia de arquitetura que visa a separação e comunicação de serviços de forma que ele se torne organizado para manutenção, a ponto de se tornar uma vantagem competitiva. Dado a alta potencialidade de interoperabilidade de microsserviços e o cruzamento de operações simultâneas, o SOA pode fazer integrações com

parceiros sem perder processos ou eficiência durante a sua programação e execução.

8. Web Service representa um termo utilizado na integração de sistemas e na comunicação entre aplicações diferentes e que possui entre suas características as possibilidades descritas abaixo, qual a alternativa incorreta? * (1 Ponto)

Possibilita a interação de novas aplicações com as já existentes.

Possibilita compatibilizar sistemas desenvolvidos em plataformas diferentes.

Devem ser codificadas na mesma linguagem de programação e utilizarem o mesmo banco de dados.

Possibilita as aplicações enviarem e receberem dados por meio de uma linguagem em formato padronizado, mensagens.

9. Sobre a arquitetura SOA- Service Oriented Architecture, podemos afirmar que: * (1 Ponto)

SOA – Arquitetura orientada a serviços é um filosofia de organização de um sistema baseada em serviços, que diminui custos, agiliza a construção de projetos e dificulta a manutenção do software.

Estilo de arquitetura para desenvolvimento de sistemas baseados em software com foco nos processos e funções de negócio da empresa, implementando-os através de serviços reutilizáveis e interoperáveis.

SOA – Arquitetura orientada a serviços permite o alto acoplamento entre as aplicações e utiliza serviços que são a menor unidade de software de uma aplicação com esse estilo de arquitetura.

Nenhuma das alternativas.

10. Na visão de engenharia de software, segundo Pressman, devemos avaliar as seguintes áreas, para criarmos uma arquitetura de software consistente para o projeto. * (1 Ponto)

1. Visão de caso de uso(negócio), visão de projeto(análise e design), visão de
2. implementação (construção), e visão de processos (fluxo de dados e lógica), visão de implantação (hardware, software e disponibilização do sistema).
3. Análise de viabilidade, matriz de rastreabilidade de requisitos e o modelo de caso de uso.
4. Diagramas de casos de uso, diagramas de classe, diagramas de sequência, e diagramas de atividades da UML.
5. Nenhuma das alternativas

TP 04

1. O que caracteriza uma arquitetura de micros serviços?

A) Um sistema composto por um único serviço centralizado

B) Um conjunto de serviços pequenos, independentes e implantáveis separadamente

- C) Um sistema que depende exclusivamente de banco de dados único
- D) Um software executado apenas em máquinas virtuais

2. Qual é um dos principais benefícios dos microserviços em ambientes de nuvem?

- A) Redução total da necessidade de testes
- B) Escalabilidade independente de serviços
- C) Eliminação de APIs
- D) Dependência de um único banco de dados

3. Em arquiteturas de microserviços, o que significa “deploy independente”?

- A) Todos os serviços devem ser implantados juntos
- B) Cada serviço pode ser atualizado sem impactar diretamente os outros
- C) O deploy só ocorre manualmente
- D) O sistema não precisa de CI/CD

4. Qual tecnologia é frequentemente utilizada para empacotar microserviços?

- A) FTP
- B) Containers
- C) Planilhas
- D) DNS

5. Qual ferramenta é amplamente usada para orquestração de containers em arquiteturas de microserviços?

- A) Excel
- B) Kubernetes
- C) Outlook
- D) Photoshop

6. Qual é uma forma comum de comunicação entre microserviços?

- A) Comunicação direta via arquivos locais
- B) APIs REST ou mensageria
- C) Apenas comunicação por e-mail
- D) Conexão direta ao banco de dados de outros serviços

7. Qual é um desafio comum em arquiteturas de microserviços?

- A) Falta de escalabilidade
- B) Gestão de comunicação entre serviços distribuídos
- C) Impossibilidade de automação
- D) Ausência de logs

8. O que é um API Gateway em microserviços?

- A) Um banco de dados distribuído
- B) Um ponto único de entrada para requisições aos serviços
- C) Um sistema de backup

D) Um servidor de arquivos

9. Em uma arquitetura de microserviços, cada serviço geralmente possui:

- A) A mesma função que todos os outros serviços
- B) Uma responsabilidade específica dentro do sistema
- C) Controle total de todo o sistema
- D) Apenas interface gráfica

10. Qual princípio é recomendado em arquiteturas de microserviços?

- A) Forte acoplamento entre serviços
- B) Baixo acoplamento e alta coesão
- C) Compartilhamento do mesmo banco de dados para todos
- D) Dependência de um único servidor

TP 05

Questionário: Arquitetura de Microserviços

1. O que caracteriza uma arquitetura de microserviços?

- A) Um sistema dividido em serviços pequenos e independentes
- B) Um único sistema monolítico com múltiplos módulos
- C) Um banco de dados centralizado para todas as funções

2. Qual é uma das principais vantagens dos microserviços?

- A) Forte acoplamento entre componentes
- B) Facilidade de escalabilidade independente
- C) Necessidade de um único deploy para todo o sistema

3. Como os microserviços normalmente se comunicam?

- A) Apenas por chamadas internas diretas
- B) Por APIs, como HTTP/REST ou mensageria
- C) Exclusivamente via banco de dados compartilhado

4. O que significa “desacoplamento” em microserviços?

- A) Serviços dependem fortemente uns dos outros
- B) Serviços são independentes e podem evoluir separadamente
- C) Todos os serviços compartilham o mesmo código

5. Qual é o papel de um API Gateway?

- A) Armazenar dados do sistema
- B) Atuar como ponto único de entrada para requisições
- C) Substituir todos os microserviços

6. Qual desafio comum em microserviços?

- A) Simplicidade na comunicação
 - B) Gerenciamento de dados distribuídos
 - C) Falta de necessidade de monitoramento
7. Qual abordagem é recomendada para bancos de dados em microserviços?
- A) Um banco de dados compartilhado por todos os serviços
 - B) Cada serviço com seu próprio banco de dados
 - C) Nenhum uso de banco de dados
8. O que é “resiliência” em microserviços?
- A) Capacidade de evitar qualquer falha
 - B) Capacidade de continuar funcionando mesmo com falhas
 - C) Eliminar a necessidade de testes
9. Qual tecnologia é frequentemente usada para comunicação assíncrona?
- A) HTML
 - B) Filas e sistemas de mensageria
 - C) Planilhas Excel
10. Por que o monitoramento é importante em microserviços?
- A) Para reduzir o número de serviços
 - B) Para identificar falhas e melhorar desempenho
 - C) Para eliminar logs

TP 06

Questionário 2 – Tipos de Armazenamento de Dados

1. Qual é a principal característica de um banco de dados relacional?
- A) Armazena dados em arquivos binários
 - B) Organiza dados em tabelas com relações entre elas
 - C) Utiliza apenas dados não estruturados
2. Qual tipo de armazenamento é mais indicado para dados altamente estruturados?
- A) NoSQL (documento)
 - B) Banco relacional
 - C) Armazenamento de blobs
3. O armazenamento de dados em formato de documento (NoSQL) geralmente utiliza:
- A) Linhas e colunas fixas
 - B) JSON, BSON ou XML
 - C) Apenas arquivos CSV

4. Qual tipo de banco NoSQL é ideal para consultas rápidas por chave única?
- A) Grafo
 - B) Chave-valor
 - C) Colunar
5. Bancos de dados de grafos são mais indicados para:
- A) Armazenamento de imagens
 - B) Relacionamentos complexos entre entidades
 - C) Dados financeiros tabulares
6. O armazenamento de blobs (Blob Storage) é mais adequado para:
- A) Dados altamente estruturados
 - B) Arquivos grandes como imagens e vídeos
 - C) Transações financeiras
7. Qual característica define bancos de dados colunares (column-family)?
- A) Dados organizados em grafos
 - B) Dados organizados por colunas em famílias
 - C) Dados apenas em formato texto
8. Quando usar armazenamento de chave-valor?
- A) Quando há necessidade de consultas complexas
 - B) Quando há acesso rápido por identificador único
 - C) Quando os dados são altamente relacionais
9. Qual tipo de armazenamento é mais flexível para mudanças no esquema de dados?
- A) Relacional
 - B) NoSQL
 - C) Data Warehouse tradicional
10. O Data Warehouse é mais indicado para:
- A) Processamento transacional (OLTP)
 - B) Análise e relatórios (OLAP)
 - C) Armazenamento de imagens
11. Qual fator é mais importante ao escolher o tipo de armazenamento?
- A) Cor da interface
 - B) Tipo de dado e padrão de acesso
 - C) Nome do banco de dados
12. Para grandes volumes de dados não estruturados, qual é a melhor

opção?

- A) Banco relacional
- B) Armazenamento de arquivos ou blobs
- C) Banco de grafos

13. Qual tipo de banco é mais adequado para redes sociais?

- A) Relacional
- B) Grafo
- C) Chave-valor

14. O que significa escalabilidade horizontal?

- A) Aumentar a capacidade de um único servidor
- B) Adicionar mais servidores ao sistema
- C) Reduzir o volume de dados

15. Qual combinação é mais adequada para aplicações modernas?

- A) Usar apenas um tipo de banco para tudo
- B) Usar múltiplos tipos de armazenamento conforme necessidade
- C) Evitar bancos NoSQL

TP 06-1

1. Os sistemas de negócios modernos gerenciam volumes cada vez maiores de dados heterogêneos. Como podemos descrever soluções que usam uma combinação de tecnologias de armazenamento de dados? (1 Ponto)

Um RDBMS tipicamente implementa um mecanismo consistente de forma transacional. Várias operações precisam ser concluídas em uma única, os dados exigem alta integridade. Os índices e as relações precisam ser mantidos com precisão. transação. O termo persistência poliglota é utilizado para descrever soluções que usam uma combinação de tecnologias.

Os dados requerem consistência forte. As transações operam de forma a garantir que todos os dados sejam 100% consistentes para todos os usuários e processos.

2. Nem todos os armazenamentos de dados em uma determinada categoria fornecem o mesmo conjunto de recursos, os bancos de dados relacionais organizam dados como uma série de tabelas bidimensionais com linhas e colunas. Quais as suas principais características quanto ao tipo de dados? (2 Pontos)

Cada chave é associada a um único valor, não há imposição do esquema, não há relações entre entidades.

Os dados podem ser gerenciados de forma desordenada, os dados de documento são semi-estruturados, o que significa que os tipos de dados de cada campo não são estritamente definidos.

As restrições são definidas no esquema e impostas no banco de dados, o tamanho das entradas de dados individuais é de pequeno a médio porte.

Os dados são armazenados em tabelas compostas por uma coluna de chave e uma ou mais famílias de colunas.

3. Um banco de dados do mecanismo de pesquisa permite que os aplicativos pesquisem informações mantidas em armazenamentos de dados externos, também fornecem suporte a análises linguísticas. Quais as características deste tipo de armazenamento? (1 Ponto)

Os índices podem ser multidimensionais e fornecer suporte para buscas de texto livre em grandes volumes de dados de texto, os Índices de dados podem ser oriundos de várias fontes e serviços, consultas são ad hoc e podem ser complexas, requer pesquisa de texto completo.

É otimizado para armazenar e recuperar grandes objetos binários (imagens, arquivos, fluxos de áudio e vídeo, grandes documentos e objetos de dados de aplicativos, imagens de disco de máquina virtual)

Executa consultas com eficiência na rede de nós e bordas e analisar as relações entre entidades. As bordas também podem ter uma direção indicando a natureza do relacionamento.

Nenhuma das alternativas.

4. Os dados históricos de várias fontes, geralmente desnormalizado em um esquema de "estrela" ou "flocos de neve", consistindo em tabelas de dimensões e fatos, geralmente é carregado com novos dados de forma programada e a fonte de dados provém de outros subsistemas. Estamos nos referindo a que tipo de armazenamento? (1 Ponto)

Banco de dados de família de coluna;

Data warehouse empresarial;

Banco de dados de gráfico;

Nenhuma das alternativas.

5. Selecionar o armazenamento de dados certo para seus requisitos é uma decisão de design chave. Literalmente, há centenas de implementações para escolher entre Banco de dados com tecnologias diferentes. Um dos pontos a serem verificados são os Requisitos Funcionais do Sistema, que estão na Visão de Caso de Uso. Quais das considerações que devemos avaliar? (1 Ponto)

Desempenho e escalabilidade, replicação, portabilidade,

Requisitos de rede, auditoria, segurança, custo geral

Formato e tamanho dos dados, simultaneidade, flexibilização, e movimentação dos dados.

Nenhuma das alternativas.

6. O arquiteto deverá analisar as condições que o sistema deverá atender, a fim de suprir todas

as expectativas do negócio e realmente agregar valor ao negócio. Os requisitos não funcionais estão diretamente associados à arquitetura de software em conjunto com o arquitetura de hardware. Assinale a alternativa correta. (1 Ponto)

Os requisitos não funcionais são condições que o sistema deve atender independente do ambiente em que é executado.

Desempenho, usabilidade, confiabilidade, performance, portabilidade devem ser avaliados de acordo com o segmento de negócio do cliente.

O custo das instalações e de gerenciamento da operação do ambiente de hardware influenciam na arquitetura de software a ser escolhida.

Nenhuma das alternativas;

7. DEVOPS foi criado para descrever um conjunto de práticas para integração entre as equipes de desenvolvimento de softwares, operações (infraestrutura ou sysadmin) e de apoio envolvidas (como controle de qualidade). Assinale a alternativa correta. (1 Ponto)

O conjunto de habilidades de sua equipe do DevOps, não interfere na escolha da arquitetura de software, e tecnologias a serem utilizadas..

A complexidade das soluções atuais não exige uma cultura DEVOPS nas empresas de tecnologia e sua constante evolução.

DEVOPS é a combinação de filosofias culturais, práticas e ferramentas que aumentam a capacidade de uma empresa de distribuir aplicativos e serviços em alta velocidade.

Nenhuma das alternativas.

8. Serviços distribuídos para fornecer acesso de dados altamente escalonável para executar operações básicas de baixo nível, controle de acesso simultâneo, permite acessar arquivos em rede. Qual o tipo de armazenamento da suporte a estes requisitos? (2 Pontos)

Banco de dados de objetos;

Banco de dados de documentos;

Banco de dados de arquivos compartilhados;

Nenhuma das alternativas.

TP 07

Questionário – Estilos de Arquitetura de Software

1. O que caracteriza a arquitetura em camadas (Layered)?

A) Uso exclusivo de APIs externas

B) Separação do sistema em níveis com responsabilidades distintas

C) Execução paralela de serviços independentes

2. Qual é a principal vantagem da arquitetura em camadas?

A) Alta escalabilidade automática

B) Facilidade de manutenção e organização

C) Eliminação de dependências

3. A arquitetura de microsserviços é caracterizada por:
- A) Um único sistema centralizado
 - B) Pequenos serviços independentes e desacoplados
 - C) Uso exclusivo de banco relacional
4. Qual é um desafio comum em microsserviços?
- A) Falta de modularidade
 - B) Gerenciamento de comunicação entre serviços
 - C) Impossibilidade de escalabilidade
5. A arquitetura orientada a eventos (Event-Driven) baseia-se em:
- A) Chamadas síncronas diretas
 - B) Produção e consumo de eventos
 - C) Processamento em lote apenas
6. Qual tecnologia é comum em arquiteturas orientadas a eventos?
- A) Filas e tópicos (message brokers)
 - B) Planilhas Excel
 - C) Arquivos locais
7. O padrão cliente-servidor define:
- A) Apenas comunicação entre bancos de dados
 - B) Separação entre quem solicita e quem fornece serviços
 - C) Execução de código em um único dispositivo
8. A arquitetura de Big Data é caracterizada principalmente por:
- A) Processamento de pequenos volumes de dados estruturados
 - B) Processamento distribuído de grandes volumes de dados variados
 - C) Uso exclusivo de bancos relacionais
9. O que caracteriza a arquitetura monolítica?
- A) Sistema dividido em vários serviços independentes
 - B) Aplicação única e integrada
 - C) Uso exclusivo de APIs REST
10. Qual é a principal desvantagem do monolito?
- A) Complexidade inicial baixa
 - B) Dificuldade de escalar e manter com o tempo
 - C) Baixo desempenho
11. A arquitetura baseada em filas (Queue-based) é útil para:
- A) Comunicação síncrona imediata

- B) Processamento assíncrono e desacoplamento
- C) Armazenamento de dados estruturados

12. Qual estilo arquitetural é mais indicado para alta escalabilidade e deploy independente?

- A) Monolítico
- B) Microsserviços
- C) Arquitetura em camadas

13. O que significa desacoplamento em arquitetura de software?

- A) Sistemas totalmente dependentes entre si
- B) Componentes com baixa dependência entre si
- C) Uso de um único banco de dados

14. Em uma arquitetura orientada a eventos, o que é um "evento"?

- A) Um erro no sistema
- B) Uma mudança de estado relevante no sistema
- C) Um banco de dados

15. Qual é a melhor abordagem para arquiteturas modernas segundo a Microsoft?

- A) Escolher um único estilo e usar sempre
- B) Combinar diferentes estilos conforme o problema
- C) Evitar arquiteturas distribuídas

16. Em arquiteturas de grande escala (large-scale), qual é um princípio fundamental?

- A) Centralização de todos os componentes
- B) Escalabilidade e tolerância a falhas
- C) Uso de apenas um servidor

4 - Provas simuladas

(Ainda estou trabalhando nisso!)

5 - Resumo de tudo

(Ainda estou trabalhando nisso!)